

Лабораторна робота №3.

Відновлення функціональних залежностей в адитивній формі у задачах розкриття концептуальної невизначеності.

Робота студента групи ПМІ-45

Хомина Віталія.

*Робота цілком повністю виконана в форматі Jupyter Notebooks,
для простоти роботи з кодом і заповненням звіту водночас.
Тому тут буде вперемішку і код, і графіки, і пояснення.*

Варіант №19.

Задана вибірка:

q_0	x_{11}	x_{12}	x_{21}	x_{22}	x_{31}	x_{32}	x_{33}	y_1	y_2	y_3	y_4
1	6.050	12.015	1.050	9.015	10.000	1.000	6.100	254.621	58.145	119.406	117.683
2	8.150	10.100	1.150	9.109	15.800	2.100	4.200	298.163	73.368	92.651	90.123
3	10.200	8.125	1.192	9.125	22.500	2.500	3.500	387.411	71.084	87.691	83.576
4	12.250	6.175	2.250	9.175	25.700	3.510	2.720	467.197	83.567	78.793	74.789
5	14.325	5.200	4.325	9.198	32.500	4.200	2.530	566.547	93.813	79.497	54.316
6	16.350	4.250	6.350	9.251	35.000	5.020	2.100	653.789	101.378	77.082	32.817
7	18.490	3.400	8.411	9.495	40.700	8.200	1.150	710.926	155.579	67.758	57.425
8	20.698	2.500	10.505	10.498	51.800	10.100	0.720	851.381	160.432	71.956	89.519
9	22.900	2.700	12.610	11.598	65.000	12.800	0.540	987.364	176.283	91.123	121.374
10	18.450	3.700	14.695	13.699	82.000	14.400	0.150	1036.123	193.657	112.859	249.173
11	16.750	4.750	15.750	15.748	95.400	14.700	0.550	1292.341	278.624	153.717	384.136
12	14.800	5.775	17.804	17.775	102.800	15.500	1.760	1088.324	354.324	117.965	479.152
13	12.950	6.800	19.850	19.798	117.000	16.300	2.230	926.939	478.926	155.912	501.239
14	10.840	7.850	18.050	21.850	125.780	16.700	3.610	877.128	588.675	169.359	625.482
15	8.910	8.855	16.910	23.855	97.000	16.900	5.160	605.327	499.367	192.924	740.976
16	15.925	10.865	14.925	25.865	95.500	17.500	8.250	458.386	468.567	218.549	875.846
17	14.929	12.885	12.011	27.875	93.900	17.700	11.370	218.859	353.932	247.354	916.124
18	13.933	14.915	10.933	25.899	91.500	18.200	13.260	195.737	335.124	284.167	863.928
19	12.935	16.950	8.935	23.951	79.580	19.100	15.510	306.168	261.946	316.375	703.153
20	11.950	18.975	6.950	21.975	55.400	19.500	17.740	685.761	151.387	341.326	631.195
21	10.810	20.995	4.950	19.015	31.500	21.000	13.140	890.639	210.519	375.651	571.588
22	8.750	22.975	2.108	17.975	12.500	23.560	11.350	923.784	485.142	344.856	436.847
23	6.150	19.950	1.251	15.950	10.800	25.300	8.580	1031.438	688.125	348.314	441.842
24	5.200	18.900	3.204	13.915	8.500	28.700	6.740	1121.321	883.435	344.716	439.425
25	4.450	17.875	5.248	11.875	4.400	31.560	4.850	1291.845	972.834	329.942	322.147

q_0	x_{11}	x_{12}	x_{21}	x_{22}	x_{31}	x_{32}	x_{33}	y_1	y_2	y_3	y_4
26	7.325	15.865	8.325	9.865	2.500	27.100	6.210	1308.614	1080.562	349.316	235.954
27	8.350	13.855	11.351	7.855	5.300	24.700	9.520	1529.956	887.987	348.231	150.492
28	9.400	11.850	15.408	5.850	8.700	26.200	10.750	1730.129	688.951	347.987	254.897
29	10.500	9.775	17.495	3.775	11.200	23.700	8.100	1917.152	455.494	342.967	458.289
30	12.600	7.750	15.607	1.750	14.700	20.360	6.100	2278.654	211.209	132.856	672.164
31	14.700	5.710	13.697	3.697	17.800	17.700	4.150	2412.145	96.197	115.632	453.356
32	16.750	3.603	11.750	5.605	20.100	13.340	2.360	2186.243	77.325	93.135	227.168
33	18.800	2.495	9.798	7.495	40.520	11.720	1.350	1862.345	64.615	77.824	106.123
34	19.850	4.394	7.850	9.415	65.200	9.900	2.130	1632.879	52.534	63.453	82.659
35	17.907	6.245	5.913	11.255	80.760	7.740	4.570	1467.156	45.178	79.167	93.834
36	15.910	8.192	3.910	13.205	91.100	6.360	6.750	1270.531	36.176	80.836	91.345
37	13.925	10.175	2.925	11.175	109.500	5.700	9.260	1084.243	20.364	87.192	96.841
38	11.929	12.125	1.929	9.125	122.900	4.750	11.790	881.956	10.428	85.834	93.952
39	9.010	14.105	3.933	7.091	108.300	3.650	13.120	616.829	8.475	101.985	109.463
40	7.935	12.010	5.935	5.985	84.500	3.520	15.360	473.329	10.924	128.591	233.415
41	5.950	10.110	7.950	3.115	58.600	2.720	12.850	249.421	24.183	102.861	308.613
42	5.020	8.115	9.995	1.115	35.800	2.340	10.340	225.356	46.324	105.817	207.319
43	4.050	6.128	11.950	2.120	15.260	2.160	8.680	176.578	76.457	78.473	182.263
44	5.935	4.131	13.935	4.130	9.520	1.760	5.320	170.948	95.814	81.417	84.132
45	6.925	2.135	15.925	6.135	4.800	1.480	2.160	168.334	104.549	78.653	61.953

Умови задачі (Варіант 9):

- b_{iq_0} визначають як середнє арифметичне значення:

$$b_{iq_0} = \frac{\left\{ \max_{i \in [1, m]} \bar{Y}_i[q_0] + \min_{i \in [1, m]} \bar{Y}_i[q_0] \right\}}{2}, q_0 = \overline{1, k_0};$$

- метод розв'язання несумісної системи рівнянь -- метод статистичного градієнта.

Хід виконання роботи:

Крок 1: імпорт необхідних бібліотек, а також допоміжні функції, які не сильно стосуються цієї лабораторної роботи.

А також визначимо алгоритм статистичного градієнта.

In [402...

```
import numpy as np
import matplotlib.pyplot as plt

def read_csv_into_matrix(file_path: str) -> np.ndarray:
    return np.genfromtxt(file_path, delimiter=",", dtype=float)
```

```
def solve_system(a: np.ndarray, b: np.ndarray) -> np.ndarray:

    if b.ndim == 1:
        b = b.reshape((b.size, 1))

    return statistic_gradient_descent(a, b, 0.001, 10000)

def statistic_gradient_descent(A, b, learning_rate, epochs):
    x = np.zeros((A.shape[1], b.shape[1]))

    for epoch in range(epochs):
        y_pred = np.dot(A, x)
        error = y_pred - b
        gradient = 2 * np.dot(A.T, error)
        x -= learning_rate * gradient

    return x
```

Крок 2: ініціалізація вхідних даних задачі

Зчитання вибірки з csv-файлу, ініціалізація параметрів;

q_0 (в програмі позначено як `sample_size`) - розмір вибірки;

n_1, n_2, n_3 - розміри векторів X_1, X_2 і X_3 відповідно;

p_1, p_2, p_3 - степені многочленів для відповідних змінних;

m - розмір вектора Y

Саме в цій частині програми задаватимуться значення цих змінних.

In [403...

```
# Variable initialization manually
sample_size: int = 45

# Y vector size
m: int = 4

# X1, X2, X3 sizes and polynomial powers. n = size, p = power
n1, n2, n3 = 2, 2, 3
p1, p2, p3 = 3, 3, 4

# Load matrix
original_matrix: np.ndarray = read_csv_into_matrix("variant_2.csv")

# this matrix will be normalized soon, so copy it instead of referencing
matrix = original_matrix[:sample_size, 1:].copy()

# create views for x1, x2, x3 and y. They reflect same changes as "matrix" does
x1 = matrix[:, :n1]
x2 = matrix[:, n1:n1+n2]
x3 = matrix[:, n1+n2:n1+n2+n3]
y = matrix[:, n1+n2+n3:]

# increase power of polynomial powers by 1 to include x^0
p1 += 1
p2 += 1
p3 += 1
```

Крок 3. Нормалізація вибірки.

Далі, необхідно нормалізувати отриману вибірку (всі значення x_{ij_i} та y_i у вибірці, для векторів X_i та Y_i відповідно).

Для того скористаємось формулою

$$\tilde{x}_{ij_i} = \frac{x_{ij_i} - d_{ij_i}^-}{d_{ij_i}^+ - d_{ij_i}^-};$$

$$\tilde{x}_{ij_i} \in [0, 1]; x_{ij_i} \in D_{ij_i}; \forall i = \overline{1, 3}$$

де $d_{ij_i}^+$ - максимальне значення x_{ij_i} серед вибірки, $d_{ij_i}^{+}$ - мінімальне значення.

Нижче присутня саме функція пошуку максимуму/мінімуму серед значень, і нормалізація всіх векторів X_i , Y_i у всіх записих вибірки.

In [404...

```
cols_min = None
cols_max = None

def normalize_matrix() -> None:
    """
    Normalize the matrix by finding column's max and min element
    then shifting the value to the left by subtracting the minimum
    and dividing the result by difference between the column's max and min
    :return:
    """
    global matrix, cols_min, cols_max

    rows, cols = matrix.shape
    cols_min = np.zeros((cols, ))
    cols_max = np.zeros((cols, ))

    for j in range(cols):
        d_min = matrix[0, j]
        d_max = matrix[0, j]
        for i in range(rows):
            if matrix[i, j] > d_max:
                d_max = matrix[i, j]
            if matrix[i, j] < d_min:
                d_min = matrix[i, j]

        cols_min[j] = d_min
        cols_max[j] = d_max

        for i in range(rows):
            matrix[i, j] -= d_min
            matrix[i, j] /= (d_max - d_min)

# normalize matrix without affecting original one
normalize_matrix()

# shortcut for y
cols_max_y = cols_max[n1+n2+n3:]
cols_min_y = cols_min[n1+n2+n3:]
```

Крок 4. Допоміжні функції - поліноми Чебишева.

Також для подальшої роботи необхідно визначити функцію, що обчислює значення многочлена Чебишева

$$T_i(x) = \begin{cases} 1, i = 0 \\ x, i = 1 \\ 2x \cdot T_{i-1}(x) - T_{i-2}(x), i \geq 2 \end{cases}$$

та зміщений многочлен Чебишева

$$\begin{cases} T_{k+1}^*(x) = T_k(2x - 1); \\ T_0^* = 0.5 \end{cases}$$

In [405...

```
def T(x: float | np.ndarray, k: int) -> float | np.ndarray:
    """
    Calculate the Chebyshev polynomial value at the given point
    :param x: polynomial argument value (as float or numpy array)
    :param k: power of the Chebyshev polynomial
    :return: float or numpy array of the Chebyshev polynomial value at given point
    """
    if k == 0:
        return 1
    elif k == 1:
        return x
    else:
        return 2 * x * T(x, k - 1) - T(x, k - 2)

def Tz(x: float | np.ndarray, k: int) -> float | np.ndarray:
    """
    Calculate the shifted Chebyshev polynomial value at the given point
    :param x: polynomial argument value (as float or numpy array)
    :param k: power of the shifted Chebyshev polynomial
    :return: float or numpy array of the shifted Chebyshev polynomial value at given point
    """
    return 0.5 if k == 0 else T(2 * x - 1, k - 1)

# because scalar and vector are mixed, vectorize the function
Tz_v = np.vectorize(Tz)
```

Крок 5: Визначення коефіцієнтів при поліномах Чебишева - матриць $||\lambda||$

Визначимо b_{iq_0} як середнє арифметичне значення:

$$b_{iq_0} = \frac{\left\{ \max_{i \in [1, m]} \bar{Y}_i[q_0] + \min_{i \in [1, m]} \bar{Y}_i[q_0] \right\}}{2}, q_0 = \overline{1, k_0};$$

А також створимо матрицю розмірністю $q_0 \times n_1(p_1 + 1) + n_2(p_2 + 1) + n_3(p_3 + 1)$ зі значеннями зміщеного полінома Чебишева. Після того, використаємо метод статистичного градієнта для визначення коефіцієнтів для тих ж многочленів.

In [406...

```
def calculate_lambda_matrix(_p1, _p2, _p3):
    def calculate_b_values():
        """
        Calculate the arithmetic mean of every Y column.
        """
        rows, cols = y.shape
        result = np.zeros(rows)

        for i in range(rows):
            min_y = y[i, 0]
            max_y = y[i, 0]

            for j in range(cols):
```

```

        if y[i, j] < min_y:
            min_y = y[i, j]
        if y[i, j] > max_y:
            max_y = y[i, j]

    result[i] = (min_y + max_y) / 2

    return result

def calculate_tz_matrix():
    """
    Calculate shifted Chebyshev polynomial values matrix
    """
    global matrix

    size, _ = matrix.shape
    _m = n1 * _p1 + n2 * _p2 + n3 * _p3

    _tz_matrix = np.zeros((size, _m))

    _tz_x1 = _tz_matrix[:, 0:(n1 * _p1)]
    _tz_x2 = _tz_matrix[:, n1 * _p1:n1 * _p1 + n2 * _p2]
    _tz_x3 = _tz_matrix[:, n1 * _p1 + n2 * _p2:n1 * _p1 + n2 * _p2 + n3 * _p3]

    for j in range(n1):
        for p in range(_p1):
            _tz_x1[:, j * _p1 + p] = Tz(x1[:, j], p)

    for j in range(n2):
        for p in range(_p2):
            _tz_x2[:, j * _p2 + p] = Tz(x2[:, j], p)

    for j in range(n3):
        for p in range(_p3):
            _tz_x3[:, j * _p3 + p] = Tz(x3[:, j], p)

    return _tz_matrix

# calculate shifted Chebyshev and mean Y's
tz_matrix: np.ndarray = calculate_tz_matrix()
b_matrix: np.ndarray = calculate_b_values()

# calculate lambdas by system of equations defined by matrices above
return solve_system(tz_matrix, b_matrix)

def split_lambdas(lambdas, _p1, _p2, _p3):
    return (
        lambdas[:n1 * _p1].reshape((n1, _p1)),
        lambdas[n1 * _p1:n1 * _p1 + n2 * _p2].reshape((n2, _p2)),
        lambdas[n1 * _p1 + n2 * _p2:n1 * _p1 + n2 * _p2 + n3 * _p3].reshape((n3, _p3))
    )

lambda_coefficients = calculate_lambda_matrix(p1, p2, p3)

# separate lambdas and reshape arrays into matrices
lambda_x1, lambda_x2, lambda_x3 = split_lambdas(lambda_coefficients, p1, p2, p3)

```

Збережемо отримані матриці $||\lambda||$ у файли, щоб потім записати у цей ж звіт:

In [407...

```

np.savetxt('lambda_x1.csv', lambda_x1, delimiter=",", fmt="%f")
np.savetxt('lambda_x2.csv', lambda_x2, delimiter=",", fmt="%f")
np.savetxt('lambda_x3.csv', lambda_x3, delimiter=",", fmt="%f")

```

Отримані результати:

Матриця $||\lambda_1||$

	0	1	2	3
X_{11}	0.021262	0.042524	-0.040394	-0.010553
X_{12}	0.021262	0.042524	0.037731	0.033616

Матриця $||\lambda_2||$

	0	1	2	3
X_{21}	0.021262	0.042524	-0.024064	0.016862
X_{22}	0.021262	0.042524	-0.042424	0.039947

Матриця $||\lambda_3||$

	0	1	2	3	4
X_{31}	0.021262	0.042524	0.013165	-0.016169	-0.007035
X_{32}	0.021262	0.042524	0.284267	-0.111493	0.010263
X_{33}	0.021262	0.042524	-0.000597	-0.057259	0.036946

Крок 6:

- Обчислення значення функції Ψ_i :

$$\Psi_{1i}(\tilde{x}_{1j_1}) = \sum_{p_1=0}^{P_1} \lambda_{1j_1p_1} T_{p_1}^*(\tilde{x}_{1j_1})$$

$$\Psi_{2i}(\tilde{x}_{2j_2}) = \sum_{p_2=0}^{P_2} \lambda_{2j_2p_2} T_{p_2}^*(\tilde{x}_{2j_2})$$

$$\Psi_{3i}(\tilde{x}_{3j_3}) = \sum_{p_3=0}^{P_3} \lambda_{3j_3p_3} T_{p_3}^*(\tilde{x}_{3j_3})$$

- а також обчислення матриці $||a||$:

$$\tilde{Y}_1(q) = \sum_{j_1=1}^{n_1} a_{1j_1}^{(i)} \Psi_{1j_1}(\tilde{x}_{1j_1}[q])$$

$$\tilde{Y}_2(q) = \sum_{j_2=1}^{n_2} a_{2j_2}^{(i)} \Psi_{2j_2}(\tilde{x}_{2j_2}[q])$$

$$\tilde{Y}_3(q) = \sum_{j_3=1}^{n_3} a_{3j_3}^{(i)} \Psi_{3j_3}(\tilde{x}_{3j_3}[q])$$

- На основі чого, обчислюємо Φ_i

$$\Phi_1(\tilde{x}_1[q]) = \sum_{j_1=1}^{n_1} a_{1j_1}^{(i)} \Psi_{1j_1}(\tilde{x}_{1j_1}[q])$$

$$\Phi_2(\tilde{x}_2[q]) = \sum_{j_2=1}^{n_2} a_{2j_2}^{(i)} \Psi_{2j_2}(\tilde{x}_{2j_2}[q])$$

$$\Phi_3(\tilde{x}_3[q]) = \sum_{j_3=1}^{n_3} a_{3j_3}^{(i)} \Psi_{3j_3}(\tilde{x}_{3j_3}[q])$$

- і отримаємо наші шукані функції наближення $\Phi_i(x_1, x_2, x_3)$

$$\Phi_i(x_1, x_2, x_3) = c_{1i}\Phi_{1i}(x_1) + c_{2i}\Phi_{2i}(x_2) + c_{3i}\Phi_{3i}(x_3)$$

- Попередньо обчисливши значення коефіцієнтів $\|c\|$ за допомогою рівняння:

$$Y_i(q) = c_{1i}\Phi_{1i}(\tilde{x}_1[q]) + c_{2i}\Phi_{2i}(\tilde{x}_2[q]) + c_{3i}\Phi_{3i}(\tilde{x}_3[q])$$

In [408...

```
def calculate_a_and_c_matrices(_lambda_x1, _lambda_x2, _lambda_x3):
    global y

    def psi(_lambda: np.ndarray, _x: np.ndarray):
        """
        Calculate Psi function values and return the matrix of it
        """
        _lambda_t = _lambda.T
        initial_value = Tz_v(_x, 0) * _lambda_t[0, :]
        for p, l in enumerate(_lambda_t[1:, :]):
            initial_value += Tz_v(_x, p+1) * l

        return initial_value

    def a(_x, _y):
        """
        Find A matrix by gradient method
        """
        return solve_system(_x, _y)

    def c(_a1, _a2, _a3, _psi_1, _psi_2, _psi_3):
        """
        Calculate C matrix
        """
        global y

        _phi_1 = _psi_1 @ _a1
        _phi_2 = _psi_2 @ _a2
        _phi_3 = _psi_3 @ _a3

        result = None

        for i in range(y.shape[1]):
            _m = np.hstack((_phi_1[:, i:i+1], _phi_2[:, i:i+1], _phi_3[:, i:i+1]))
            col = solve_system(_m, y[:, i:i+1])

            if result is None:
                result = col
            else:
                result = np.hstack((result, col))

        return result

    def recalculate_ys(_c_matrix, _a1, _a2, _a3, _psi_1, _psi_2, _psi_3):
        """
        Recalculate Y vectors on given C and A matrices, to compare with original
        """
```

```

rows, cols = y.shape
result = np.zeros(y.shape)

_phi_1 = _psi_1 @ _a1
_phi_2 = _psi_2 @ _a2
_phi_3 = _psi_3 @ _a3

_c_t = _c_matrix.T

for i in range(rows):
    for j in range(cols):
        s1 = _phi_1[i, :] * _c_t[:, 0]
        s2 = _phi_2[i, :] * _c_t[:, 1]
        s3 = _phi_3[i, :] * _c_t[:, 2]
        result[i, :] = s1 + s2 + s3

return result

psi_1 = psi(_lambda_x1, x1)
psi_2 = psi(_lambda_x2, x2)
psi_3 = psi(_lambda_x3, x3)

_a1_matrix = a(psi_1, y)
_a2_matrix = a(psi_2, y)
_a3_matrix = a(psi_3, y)

_c_matrix = c(_a1_matrix, _a2_matrix, _a3_matrix, psi_1, psi_2, psi_3)
_recalculated_ys = recalculate_ys(_c_matrix, _a1_matrix, _a2_matrix, _a3_matrix, psi_1, p

return _a1_matrix, _a2_matrix, _a3_matrix, _c_matrix, _recalculated_ys

a1_matrix, a2_matrix, a3_matrix, c_matrix, recalculated_ys = (
    calculate_a_and_c_matrices(lambda_x1, lambda_x2, lambda_x3)
)

```

In [409...

```

np.savetxt("a1_matrix.csv", a1_matrix, delimiter=",", fmt="%f")
np.savetxt("a2_matrix.csv", a2_matrix, delimiter=",", fmt="%f")
np.savetxt("a3_matrix.csv", a3_matrix, delimiter=",", fmt="%f")

np.savetxt("c_matrix.csv", c_matrix, delimiter=",", fmt="%f")

```

Отримані результати

Матриця A_1 (Подано в табличній формі, стовпці і рядки підписані відповідними змінними/ векторами + транспоновано)

	Y_1	Y_2	Y_3	Y_4
X_{11}	3.418130	2.949648	3.879473	3.749669
X_{12}	2.302165	1.962932	3.707619	2.017105

Матриця A_2

	Y_1	Y_2	Y_3	Y_4
X_{21}	3.128226	3.045584	4.468580	3.094547
X_{22}	2.915724	0.868061	1.263200	2.056138

Матриця A_3

	Y_1	Y_2	Y_3	Y_4
X_{31}	2.376929	0.656962	1.714606	3.169176
X_{32}	0.727560	0.910063	1.243827	0.915544
X_{33}	2.330024	2.456702	2.818642	1.732470

Матриця коефіцієнтів C

	Y_1	Y_2	Y_3	Y_4
X_1	-0.945239	0.928338	1.363188	0.541826
X_2	0.893916	-0.867815	-1.148354	-0.497368
X_3	1.103078	0.783216	0.580392	0.892954

Крок 7: відновлення функцій

Тепер пора поступово відновити вигляд функціональних залежностей (поки в нормованому вигляді). Почнемо з $\Phi(x_1, x_2, x_3)$, де використовуємо коефіцієнти з матриці C : (тут обійдемося прямим виводом з коду програми)

```
In [410... def print_overall_phi_functions(_c_matrix):
    rows, cols = _c_matrix.shape
    for j in range(cols):
        s = f"Φ{j+1} = " + " + ".join([f"{_c_matrix[i, j]}*Φ{j+1}{i+1}" for i in range(rows)])
        print(s)

print_overall_phi_functions(c_matrix)
```

```
Φ1 = -0.9452388051692961*Φ11 + 0.893915936949392*Φ12 + 1.103077916394471*Φ13
Φ2 = 0.9283381457126668*Φ21 + -0.867815323249241*Φ22 + 0.7832163849746987*Φ23
Φ3 = 1.3631883187683151*Φ31 + -1.1483542356934187*Φ32 + 0.5803917012044099*Φ33
Φ4 = 0.5418259140117551*Φ41 + -0.49736751157754666*Φ42 + 0.8929544308945925*Φ43
```

Тепер виразимо всі функції

$$\Phi_{ik}(\tilde{x}_k) = \sum_{j_k=1}^{n_k} \sum_{p_k=0}^{P_k} a_{kj_k}^{(i)} \lambda_{kj_k p_k} T_{p_k}^*(\tilde{x}_{kj_k})$$

(Знову, через програмний вивід)

```
In [411... def print_all_phis(
    _a1_matrix, _a2_matrix, _a3_matrix,
    _lambda_x1, _lambda_x2, _lambda_x3,
):
    global m

    def print_phi_function(first_index: str, index: int, a_matrix: np.ndarray, _lambda_x: np.ndarray):
        global m

        return f"Φ{first_index}{index+1}(x) = " + (
            " + \n " .join(
                [
                    " + ".join([
                        f"{row * a_col :.8f} T*({x}^{a_idx})"
                        for a_idx, a_col in enumerate(_lambda_x[idx, :])
                    ]) for idx, row in enumerate(a_matrix[:, index])
                ]
            )
        )
```

```

    )

lst = [(1, _lambda_x1, _a1_matrix), (2, _lambda_x2, _a2_matrix), (3, _lambda_x3, _a3_matrix)]
for fi, l, a in lst:
    for si in range(0, m):
        print(print_phi_function(fi, si, a, l))

print_all_phis(
    a1_matrix, a2_matrix, a3_matrix,
    lambda_x1, lambda_x2, lambda_x3,
)

```

```

phi11(x) = 0.07267552 T*(x^0) + 0.14535104 T*(x^1) + -0.13807047 T*(x^2) + -0.03607070 T*(x^3) +
0.04894812 T*(x^0) + 0.09789624 T*(x^1) + 0.08686373 T*(x^2) + 0.07738852 T*(x^3)
phi12(x) = 0.06271475 T*(x^0) + 0.12542950 T*(x^1) + -0.11914679 T*(x^2) + -0.03112691 T*(x^3) +
0.04173543 T*(x^0) + 0.08347085 T*(x^1) + 0.07406403 T*(x^2) + 0.06598502 T*(x^3)
phi13(x) = 0.08248450 T*(x^0) + 0.16496900 T*(x^1) + -0.15670577 T*(x^2) + -0.04093914 T*(x^3) +
0.07883056 T*(x^0) + 0.15766112 T*(x^1) + 0.13989336 T*(x^2) + 0.12463360 T*(x^3)
phi14(x) = 0.07972463 T*(x^0) + 0.15944925 T*(x^1) + -0.15146251 T*(x^2) + -0.03956935 T*(x^3) +
0.04288723 T*(x^0) + 0.08577446 T*(x^1) + 0.07610802 T*(x^2) + 0.06780606 T*(x^3)
phi21(x) = 0.06651165 T*(x^0) + 0.13302331 T*(x^1) + -0.07527837 T*(x^2) + 0.05274781 T*(x^3) +
0.06199348 T*(x^0) + 0.12398695 T*(x^1) + -0.12369561 T*(x^2) + 0.11647451 T*(x^3)
phi22(x) = 0.06475453 T*(x^0) + 0.12950907 T*(x^1) + -0.07328965 T*(x^2) + 0.05135430 T*(x^3) +
0.01845651 T*(x^0) + 0.03691302 T*(x^1) + -0.03682628 T*(x^2) + 0.03467644 T*(x^3)
phi23(x) = 0.09500994 T*(x^0) + 0.19001988 T*(x^1) + -0.10753294 T*(x^2) + 0.07534869 T*(x^3) +
0.02685787 T*(x^0) + 0.05371573 T*(x^1) + -0.05358951 T*(x^2) + 0.05046107 T*(x^3)
phi24(x) = 0.06579557 T*(x^0) + 0.13159115 T*(x^1) + -0.07446791 T*(x^2) + 0.05217991 T*(x^3) +
0.04371715 T*(x^0) + 0.08743430 T*(x^1) + -0.08722885 T*(x^2) + 0.08213660 T*(x^3)
phi31(x) = 0.05053772 T*(x^0) + 0.10107545 T*(x^1) + 0.03129133 T*(x^2) + -0.03843340 T*(x^3) +
-0.01672180 T*(x^4) +
0.01546922 T*(x^0) + 0.03093844 T*(x^1) + 0.20682109 T*(x^2) + -0.08111796 T*(x^3) + 0.007466
59 T*(x^4) +
0.04954046 T*(x^0) + 0.09908091 T*(x^1) + -0.00139065 T*(x^2) + -0.13341580 T*(x^3) + 0.08608
485 T*(x^4)
phi32(x) = 0.01396819 T*(x^0) + 0.02793638 T*(x^1) + 0.00864865 T*(x^2) + -0.01062266 T*(x^3) +
-0.00462176 T*(x^4) +
0.01934956 T*(x^0) + 0.03869912 T*(x^1) + 0.25870059 T*(x^2) + -0.10146579 T*(x^3) + 0.009339
53 T*(x^4) +
0.05223386 T*(x^0) + 0.10446771 T*(x^1) + -0.00146625 T*(x^2) + -0.14066930 T*(x^3) + 0.09076
508 T*(x^4)
phi33(x) = 0.03645558 T*(x^0) + 0.07291116 T*(x^1) + 0.02257212 T*(x^2) + -0.02772408 T*(x^3) +
-0.01206234 T*(x^4) +
0.02644598 T*(x^0) + 0.05289196 T*(x^1) + 0.35357863 T*(x^2) + -0.13867821 T*(x^3) + 0.012764
78 T*(x^4) +
0.05992933 T*(x^0) + 0.11985866 T*(x^1) + -0.00168227 T*(x^2) + -0.16139374 T*(x^3) + 0.10413
726 T*(x^4)
phi34(x) = 0.06738231 T*(x^0) + 0.13476461 T*(x^1) + 0.04172095 T*(x^2) + -0.05124352 T*(x^3) +
-0.02229530 T*(x^4) +
0.01946609 T*(x^0) + 0.03893218 T*(x^1) + 0.26025860 T*(x^2) + -0.10207686 T*(x^3) + 0.009395
77 T*(x^4) +
0.03683540 T*(x^0) + 0.07367079 T*(x^1) + -0.00103400 T*(x^2) + -0.09920021 T*(x^3) + 0.06400
768 T*(x^4)

```

Крок 8. Обчислення нев'язки, зображення графів.

Нев'язку порахуємо як середнє квадратичне різниць між реальним значенням вибірки і обчисленим для кожного значення вибірки, задля рівномірності.

In [412...

```

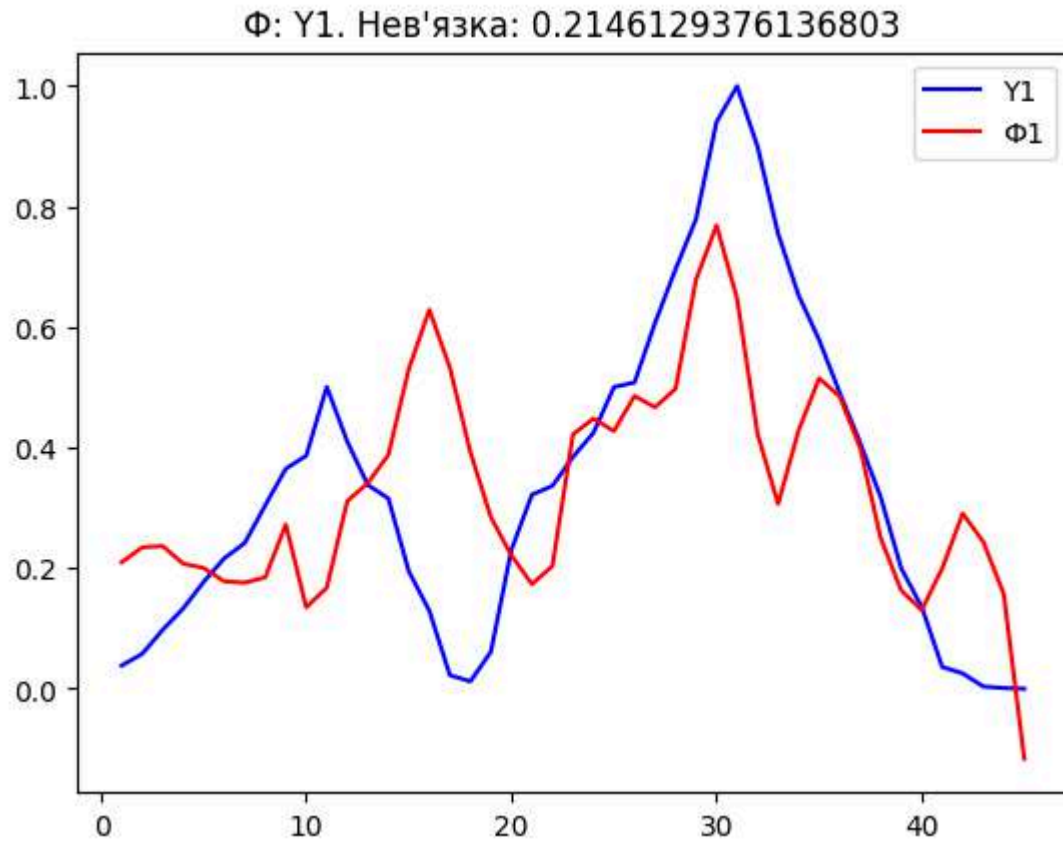
def incoherence(idx, elaluated, real):
    _m = real.shape[0]
    return np.sqrt(np.sum(np.power(elaluated[:, idx] - real[:, idx], 2)) / _m)

```

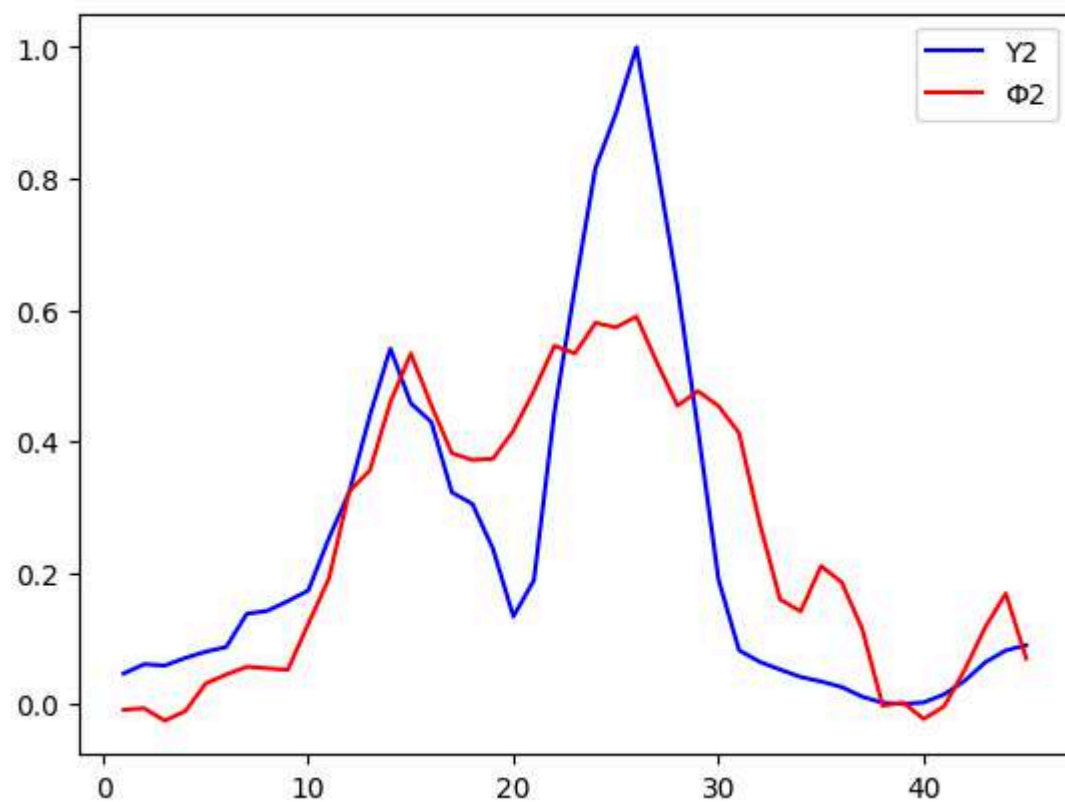
```
def plot_all(evaluated, real):
    global m
    for _i in range(m):
        plt.title(f"Φ: Y{_i+1}. Нев'язка: {incoherence(_i, evaluated, real)}")
        plt.plot(range(1, sample_size+1), real[:, _i], color='blue', label=f"Y{_i+1}")
        plt.plot(range(1, sample_size+1), evaluated[:, _i], color='red', label=f"Φ{_i+1}")
        plt.legend(loc='upper right')
        plt.show()

print("Поточні степені поліномів: ", (p1, p2, p3))
plot_all(recalculated_ys, y)
```

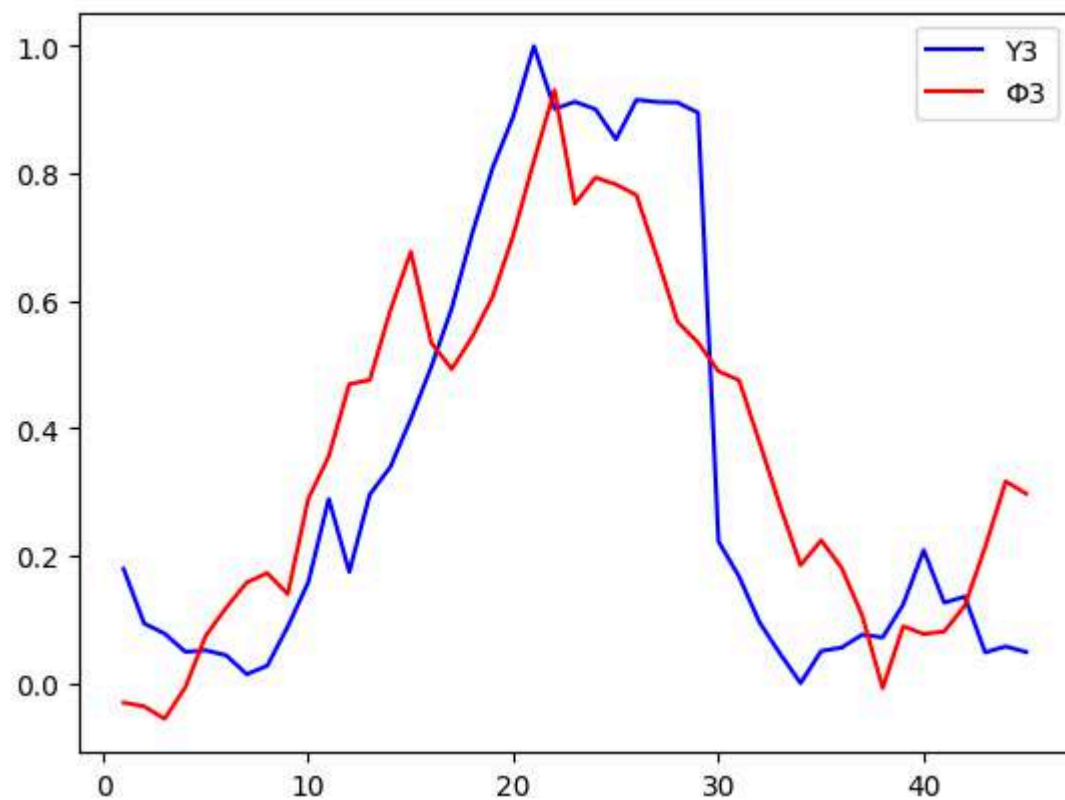
Поточні степені поліномів: (4, 4, 5)



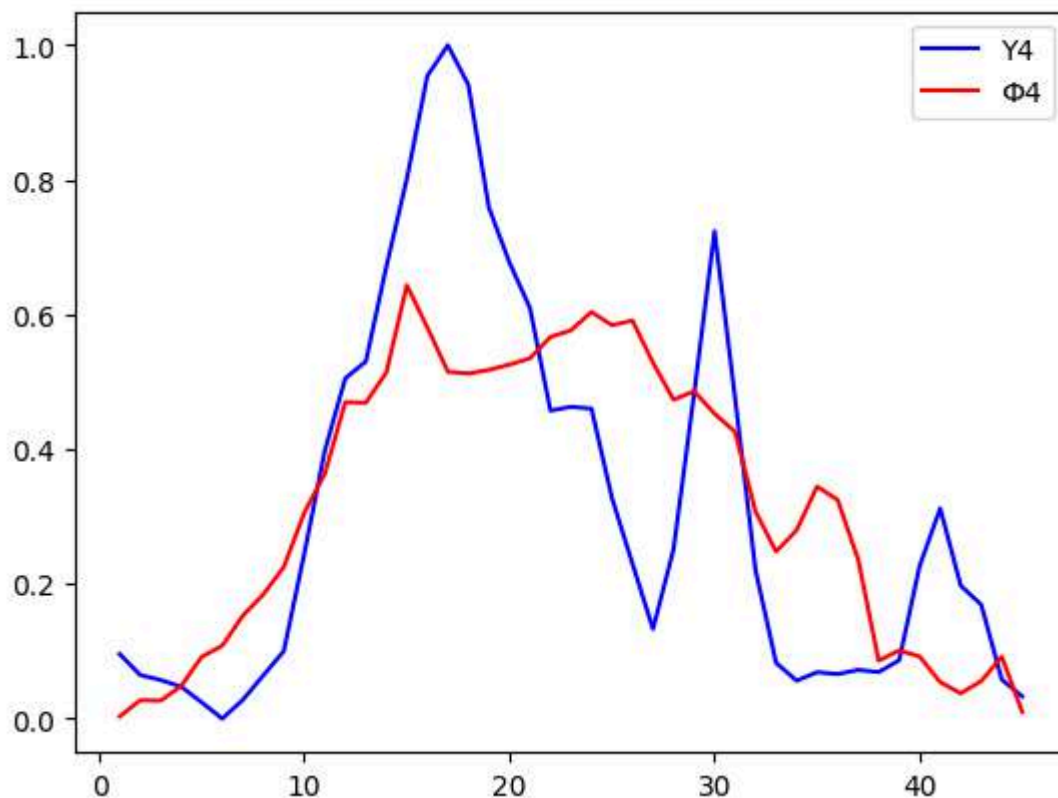
Ф: Y2. Нев'язка: 0.15405522513254896



Ф: Y3. Нев'язка: 0.18172330891497104



Ф: Y4. Нев'язка: 0.19325401194577474



Крок 9. Спроба знайти функцію за мінімальної (максимальної серед Φ_i) нев'язки

Тепер проробимо кроки 1-8, перебираючи P_1 , P_2 , P_3 степені полінома (щонайбільше до 9), таким чином, щоб максимальна нев'язка серед функцій була мінімальною. Як виясимо нижче, це будуть степені 5, 3, 5 відповідно.

In [413...

```
# This cell is ugly code
def find_minimal_incoherence():
    global y, m, n1, n2, n3

    def _try(_p1, _p2, _p3):
        _lambda_coefficients = calculate_lambda_matrix(_p1, _p2, _p3)
        _l1, _l2, _l3 = split_lambdas(_lambda_coefficients, _p1, _p2, _p3)

        _a1, _a2, _a3, _c, evaluated = calculate_a_and_c_matrices(_l1, _l2, _l3)
        return max(incoherence(idx, evaluated, y) for idx in range(m)), _a1, _a2, _a3, _c, ev

    curr_incoherence = float('+inf')
    curr_pows = 0, 0, 0
    curr_rest = None
    for _p1 in range(3, 7):
        for _p2 in range(3, 7):
            for _p3 in range(3, 7):
                max_c, *rest = _try(_p1, _p2, _p3)
                if max_c < curr_incoherence:
                    curr_incoherence = max_c
                    curr_pows = _p1, _p2, _p3
                    curr_rest = rest
    return curr_pows, curr_incoherence, *curr_rest

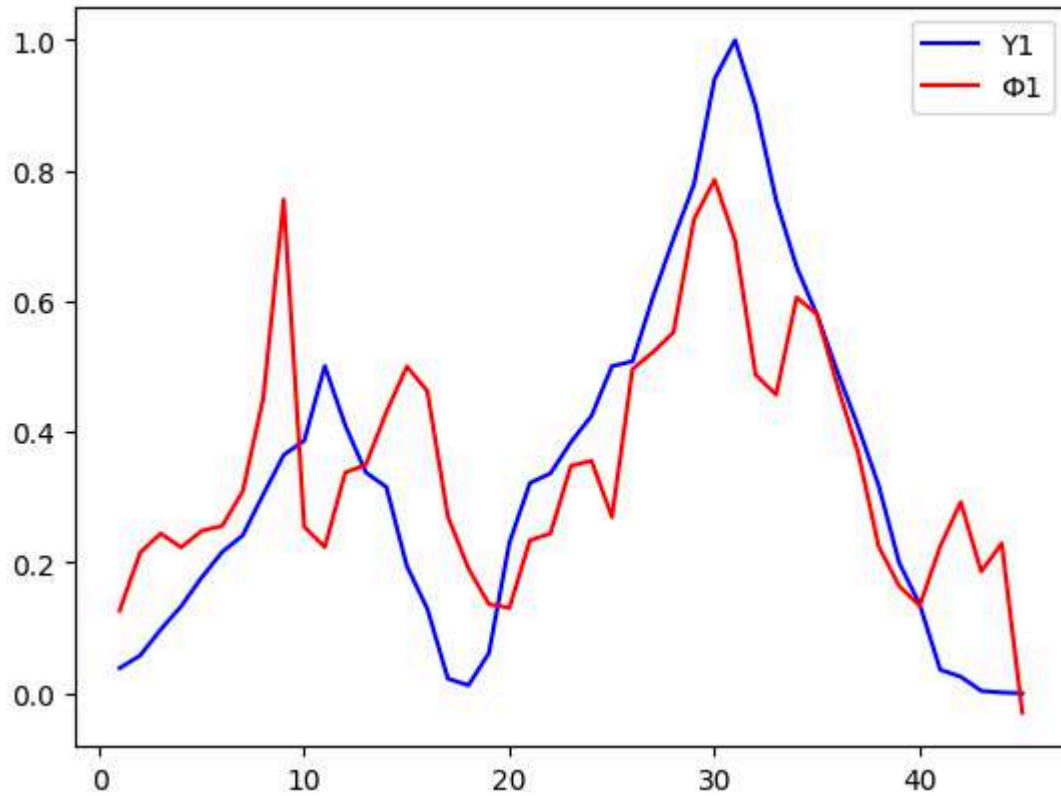
cp, ci, a1, a2, a3, c, e, l1, l2, l3 = find_minimal_incoherence()
print(f"Мінімальна нев'язка: {ci}, при {cp}")

plot_all(e, y)
print()
```

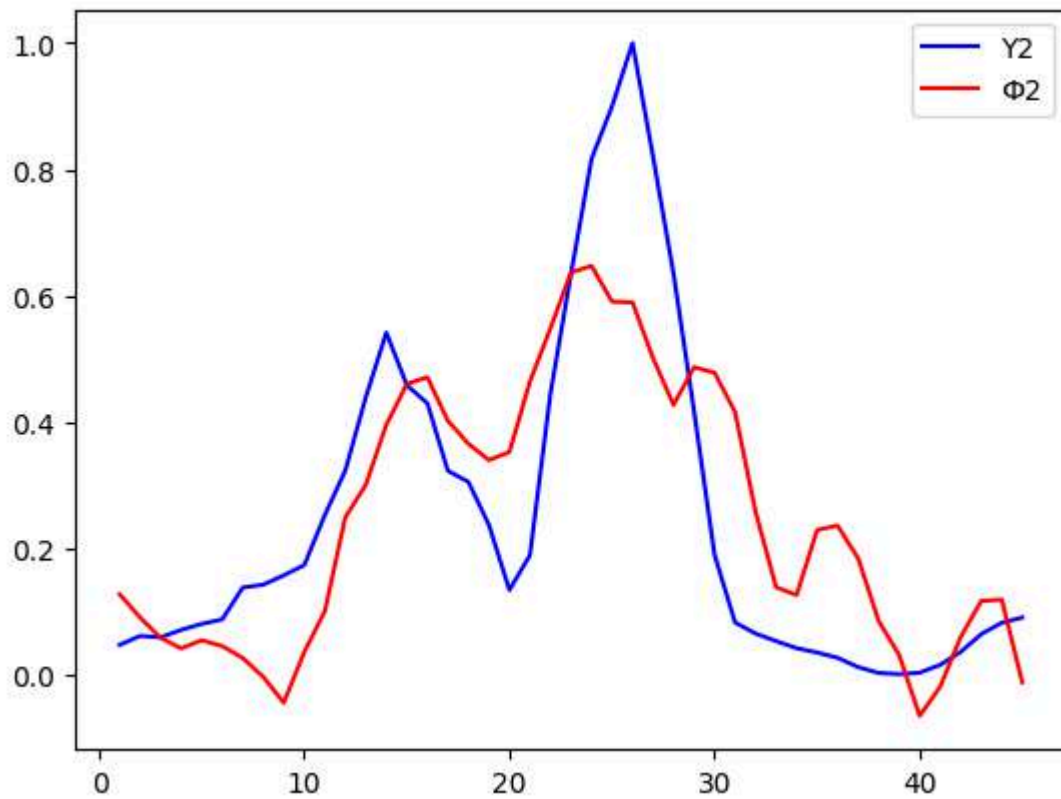
```
print_overall_phi_functions(c)
print()
print_all_phis(a1, a2, a3, l1, l2, l3)
```

Мінімальна нев'язка: 0.17640156184980058, при (5, 3, 5)

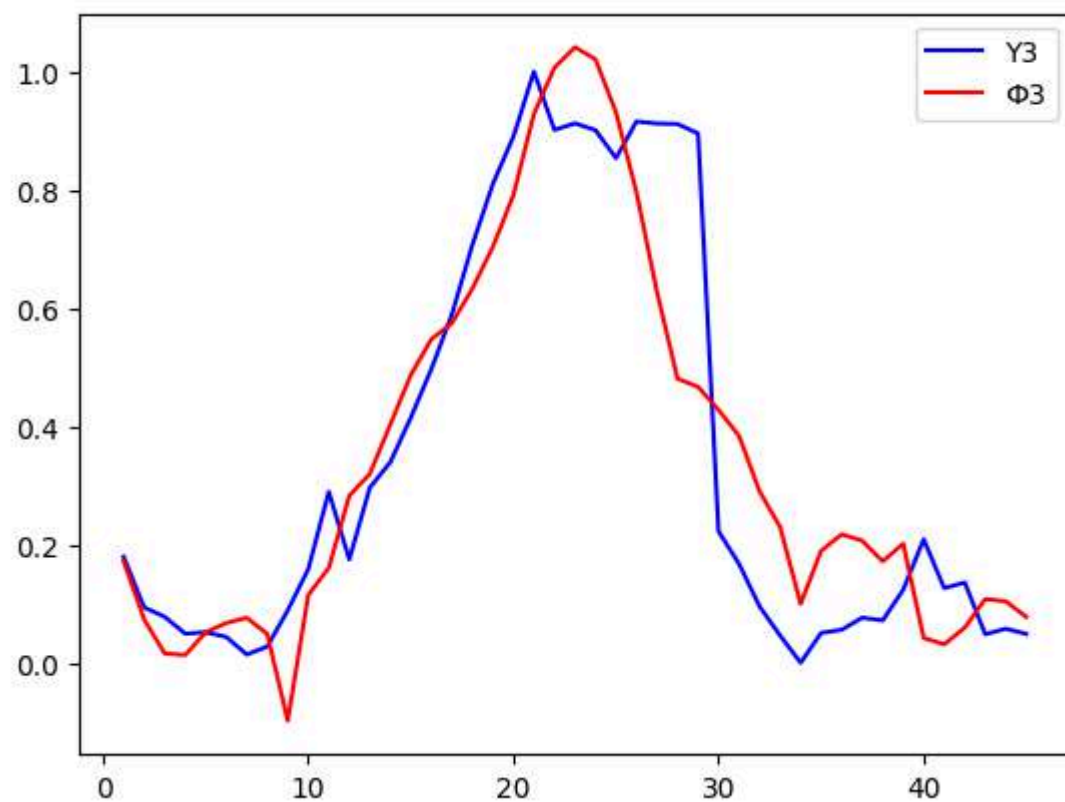
Ф: Y1. Нев'язка: 0.17411745181872962



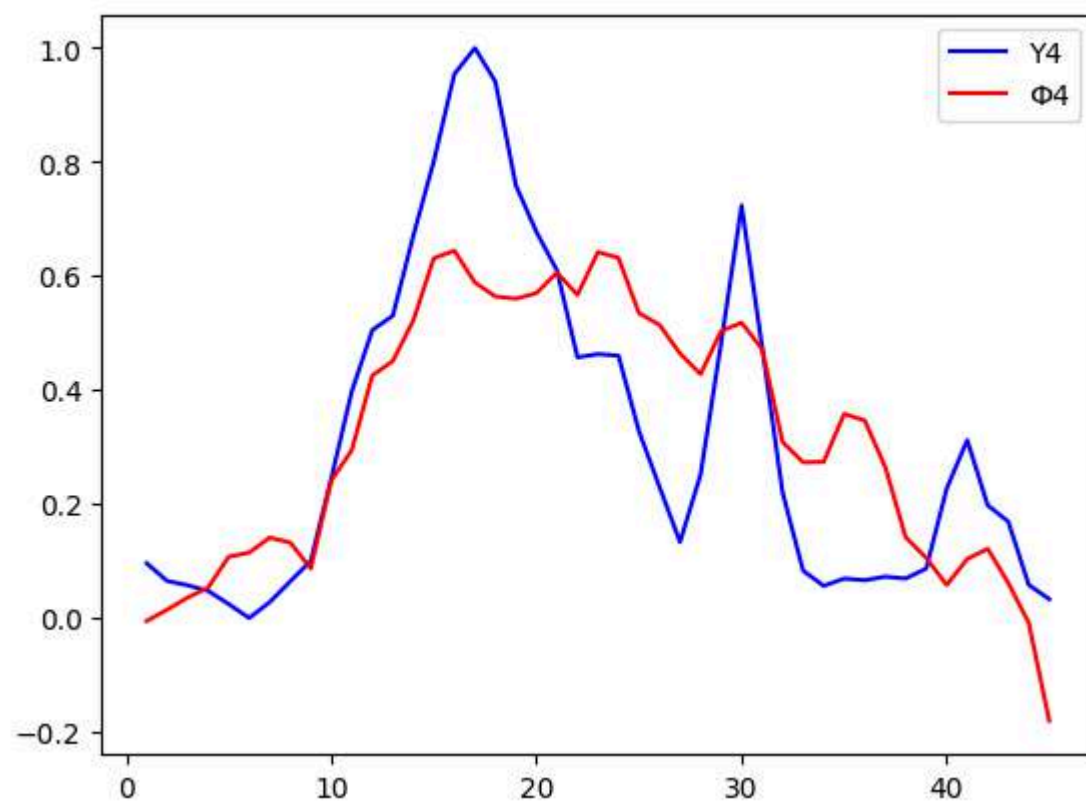
Ф: Y2. Нев'язка: 0.16050147286232683



Ф: Y3. Нев'язка: 0.14386687010562754



Ф: Y4. Нев'язка: 0.17640156184980058



$\Phi_1 = -1.2953296767137352 \cdot \Phi_{11} + 1.2231319505079974 \cdot \Phi_{12} + 1.0254568371153128 \cdot \Phi_{13}$
 $\Phi_2 = 0.5050367607369766 \cdot \Phi_{21} + -0.22732331289592442 \cdot \Phi_{22} + 0.7839513413719778 \cdot \Phi_{23}$
 $\Phi_3 = 0.9710745425826631 \cdot \Phi_{31} + -0.6179139714558767 \cdot \Phi_{32} + 0.595600300499316 \cdot \Phi_{33}$
 $\Phi_4 = 0.5561939596383446 \cdot \Phi_{41} + -0.6037567127567985 \cdot \Phi_{42} + 0.9706924726103208 \cdot \Phi_{43}$

$\Phi_{11}(x) = 0.08325538 T^*(x^0) + 0.16651076 T^*(x^1) + -0.20514066 T^*(x^2) + -0.07519705 T^*(x^3) +$
 $-0.06839407 T^*(x^4) +$
 $0.01575771 T^*(x^0) + 0.03151543 T^*(x^1) + 0.04590960 T^*(x^2) + 0.01982002 T^*(x^3) + -0.012677$
 $65 T^*(x^4)$
 $\Phi_{12}(x) = 0.05707781 T^*(x^0) + 0.11415563 T^*(x^1) + -0.14063932 T^*(x^2) + -0.05155322 T^*(x^3) +$
 $-0.04688927 T^*(x^4) +$
 $0.05169893 T^*(x^0) + 0.10339786 T^*(x^1) + 0.15062319 T^*(x^2) + 0.06502680 T^*(x^3) + -0.041593$
 $64 T^*(x^4)$
 $\Phi_{13}(x) = 0.07239560 T^*(x^0) + 0.14479120 T^*(x^1) + -0.17838224 T^*(x^2) + -0.06538839 T^*(x^3) +$
 $-0.05947279 T^*(x^4) +$
 $0.10344705 T^*(x^0) + 0.20689411 T^*(x^1) + 0.30138969 T^*(x^2) + 0.13011548 T^*(x^3) + -0.083226$
 $86 T^*(x^4)$
 $\Phi_{14}(x) = 0.07977239 T^*(x^0) + 0.15954477 T^*(x^1) + -0.19655858 T^*(x^2) + -0.07205118 T^*(x^3) +$
 $-0.06553280 T^*(x^4) +$
 $0.04082503 T^*(x^0) + 0.08165005 T^*(x^1) + 0.11894241 T^*(x^2) + 0.05134963 T^*(x^3) + -0.032845$
 $20 T^*(x^4)$
 $\Phi_{21}(x) = 0.05548359 T^*(x^0) + 0.11096718 T^*(x^1) + 0.00327631 T^*(x^2) +$
 $0.06364927 T^*(x^0) + 0.12729854 T^*(x^1) + -0.17485915 T^*(x^2)$
 $\Phi_{22}(x) = 0.06265874 T^*(x^0) + 0.12531748 T^*(x^1) + 0.00370001 T^*(x^2) +$
 $0.01881773 T^*(x^0) + 0.03763546 T^*(x^1) + -0.05169663 T^*(x^2)$
 $\Phi_{23}(x) = 0.09265581 T^*(x^0) + 0.18531162 T^*(x^1) + 0.00547134 T^*(x^2) +$
 $0.01985163 T^*(x^0) + 0.03970327 T^*(x^1) + -0.05453699 T^*(x^2)$
 $\Phi_{24}(x) = 0.09748698 T^*(x^0) + 0.19497395 T^*(x^1) + 0.00575662 T^*(x^2) +$
 $0.00389427 T^*(x^0) + 0.00778854 T^*(x^1) + -0.01069845 T^*(x^2)$
 $\Phi_{31}(x) = 0.04629969 T^*(x^0) + 0.09259939 T^*(x^1) + 0.06661529 T^*(x^2) + -0.07052842 T^*(x^3) +$
 $0.01569208 T^*(x^4) +$
 $0.01517145 T^*(x^0) + 0.03034291 T^*(x^1) + 0.20446697 T^*(x^2) + -0.08590596 T^*(x^3) + -0.01170$
 $989 T^*(x^4) +$
 $0.04366145 T^*(x^0) + 0.08732290 T^*(x^1) + -0.03172635 T^*(x^2) + -0.17824857 T^*(x^3) + 0.09285$
 $250 T^*(x^4)$
 $\Phi_{32}(x) = 0.00487561 T^*(x^0) + 0.00975122 T^*(x^1) + 0.00701495 T^*(x^2) + -0.00742703 T^*(x^3) +$
 $0.00165246 T^*(x^4) +$
 $0.01889407 T^*(x^0) + 0.03778814 T^*(x^1) + 0.25463698 T^*(x^2) + -0.10698468 T^*(x^3) + -0.01458$
 $314 T^*(x^4) +$
 $0.04499625 T^*(x^0) + 0.08999251 T^*(x^1) + -0.03269628 T^*(x^2) + -0.18369794 T^*(x^3) + 0.09569$
 $116 T^*(x^4)$
 $\Phi_{33}(x) = 0.02722228 T^*(x^0) + 0.05444455 T^*(x^1) + 0.03916700 T^*(x^2) + -0.04146776 T^*(x^3) +$
 $0.00922629 T^*(x^4) +$
 $0.02686887 T^*(x^0) + 0.05373774 T^*(x^1) + 0.36211404 T^*(x^2) + -0.15214073 T^*(x^3) + -0.02073$
 $838 T^*(x^4) +$
 $0.05047485 T^*(x^0) + 0.10094970 T^*(x^1) + -0.03667727 T^*(x^2) + -0.20606440 T^*(x^3) + 0.10734$
 $220 T^*(x^4)$
 $\Phi_{34}(x) = 0.06589418 T^*(x^0) + 0.13178835 T^*(x^1) + 0.09480753 T^*(x^2) + -0.10037674 T^*(x^3) +$
 $0.02233312 T^*(x^4) +$
 $0.02045714 T^*(x^0) + 0.04091428 T^*(x^1) + 0.27570259 T^*(x^2) + -0.11583532 T^*(x^3) + -0.01578$
 $957 T^*(x^4) +$
 $0.03232376 T^*(x^0) + 0.06464751 T^*(x^1) + -0.02348788 T^*(x^2) + -0.13196226 T^*(x^3) + 0.06874$
 $123 T^*(x^4)$

Крок 11. Денормалізація коефіцієнтів

$$\tilde{x}_{ij_i} = \frac{x_{ij_i} - d_{ij_i}^-}{d_{ij_i}^+ - d_{ij_i}^-} \implies x_{ij_i} = \tilde{x}_{ij_i} * (d_{ij_i}^+ - d_{ij_i}^-) + d_{ij_i}^-$$

In [414...

```
def denormalize(a_matrix):
    rows, cols = a_matrix.shape
    result = np.zeros((rows, cols))
    for j in range(cols):
        for i in range(rows):
```

```

result[i, j] = a_matrix[i, j]* (cols_max_y[j] - cols_min_y[j]) + cols_min[j]
return result

# cp, ci, a1, a2, a3, c, e, l1, l2, l3 = find_minimal_incoherence()
denormalized_a1 = denormalize(a1)
denormalized_a2 = denormalize(a2)
denormalized_a3 = denormalize(a3)

denormalized_c = denormalize(c)
print_overall_phi_functions(denormalized_c)
print()
print_all_phis(denormalized_a1, denormalized_a2, denormalized_a3, l1, l2, l3)

```

```

Φ1 = -2902.424977236723*Φ11 + 2748.5269250013007*Φ12 + 2304.9813311445473*Φ13
Φ2 = 543.578345708223*Φ21 + -241.57536855265292*Φ22 + 842.5990417174595*Φ23
Φ3 = 304.21753004522225*Φ31 + -191.86150606058177*Φ32 + 186.99522261528546*Φ33
Φ4 = 492.4050179062673*Φ41 + -532.1875306750694*Φ42 + 858.5344559040046*Φ43

```

```

Φ11(x) = 186.88842968 T*(x^0) + 373.77685936 T*(x^1) + -460.49174705 T*(x^2) + -168.79940377 T
*(x^3) + -153.52834960 T*(x^4) +
35.43642081 T*(x^0) + 70.87284162 T*(x^1) + 103.24288315 T*(x^2) + 44.57185402 T*(x^3) + -28.
50987096 T*(x^4)
Φ12(x) = 61.23407454 T*(x^0) + 122.46814908 T*(x^1) + -150.88031941 T*(x^2) + -55.30719741 T*
(x^3) + -50.30362992 T*(x^4) +
55.46744359 T*(x^0) + 110.93488717 T*(x^1) + 161.60262988 T*(x^2) + 69.76683147 T*(x^3) + -4
4.62554692 T*(x^4)
Φ13(x) = 22.62226592 T*(x^0) + 45.24453185 T*(x^1) + -55.74110059 T*(x^2) + -20.43264533 T*(x^
3) + -18.58413149 T*(x^4) +
32.31646790 T*(x^0) + 64.63293579 T*(x^1) + 94.15299972 T*(x^2) + 40.64758394 T*(x^3) + -25.9
9975699 T*(x^4)
Φ14(x) = 70.48528030 T*(x^0) + 140.97056060 T*(x^1) + -173.67522389 T*(x^2) + -63.66297427 T*
(x^3) + -57.90347093 T*(x^4) +
36.08280498 T*(x^0) + 72.16560996 T*(x^1) + 105.12610284 T*(x^2) + 45.38487465 T*(x^3) + -29.
02991020 T*(x^4)
Φ21(x) = 124.57377483 T*(x^0) + 249.14754966 T*(x^1) + 7.35610066 T*(x^2) +
142.89601785 T*(x^0) + 285.79203570 T*(x^1) + -392.56816222 T*(x^2)
Φ22(x) = 67.21731331 T*(x^0) + 134.43462661 T*(x^1) + 3.96919274 T*(x^2) +
20.21593868 T*(x^0) + 40.43187736 T*(x^1) + -55.53782404 T*(x^2)
Φ23(x) = 28.94746283 T*(x^0) + 57.89492567 T*(x^1) + 1.70935215 T*(x^2) +
6.21814482 T*(x^0) + 12.43628964 T*(x^1) + -17.08267117 T*(x^2)
Φ24(x) = 86.13270192 T*(x^0) + 172.26540384 T*(x^1) + 5.08614936 T*(x^2) +
3.46160935 T*(x^0) + 6.92321869 T*(x^1) + -9.50983548 T*(x^2)
Φ31(x) = 103.96684998 T*(x^0) + 207.93369995 T*(x^1) + 149.58591755 T*(x^2) + -158.37293139 T*
(x^3) + 35.23686818 T*(x^4) +
34.12096505 T*(x^0) + 68.24193010 T*(x^1) + 459.85113624 T*(x^2) + -193.20457492 T*(x^3) + -2
6.33581745 T*(x^4) +
98.04712636 T*(x^0) + 196.09425271 T*(x^1) + -71.24540340 T*(x^2) + -400.27898729 T*(x^3) + 2
08.51165196 T*(x^4)
Φ32(x) = 5.26876966 T*(x^0) + 10.53753933 T*(x^1) + 7.58062541 T*(x^2) + -8.02592842 T*(x^3) +
1.78571287 T*(x^4) +
20.29777926 T*(x^0) + 40.59555852 T*(x^1) + 273.55489042 T*(x^2) + -114.93296886 T*(x^3) + -1
5.66657357 T*(x^4) +
48.28159254 T*(x^0) + 96.56318508 T*(x^1) + -35.08355283 T*(x^2) + -197.11038645 T*(x^3) + 10
2.67791616 T*(x^4)
Φ33(x) = 8.51924528 T*(x^0) + 17.03849057 T*(x^1) + 12.25736014 T*(x^2) + -12.97738509 T*(x^3)
+ 2.88737730 T*(x^4) +
8.40891216 T*(x^0) + 16.81782433 T*(x^1) + 113.32762151 T*(x^2) + -47.61413687 T*(x^3) + -6.4
9030810 T*(x^4) +
15.77865225 T*(x^0) + 31.55730451 T*(x^1) + -11.46547060 T*(x^2) + -64.41660433 T*(x^3) + 33.
55562748 T*(x^4)
Φ34(x) = 58.22656092 T*(x^0) + 116.45312184 T*(x^1) + 83.77548750 T*(x^2) + -88.69664840 T*(x^
3) + 19.73438314 T*(x^4) +
18.09170782 T*(x^0) + 36.18341564 T*(x^1) + 243.82347880 T*(x^2) + -102.44143781 T*(x^3) + -1
3.96384639 T*(x^4) +
28.57357419 T*(x^0) + 57.14714837 T*(x^1) + -20.76283003 T*(x^2) + -116.65208113 T*(x^3) + 6
0.76591306 T*(x^4)

```

